
LLM-Lasso: A Robust Framework for Domain-Informed Feature Selection and Regularization

Erica Zhang^{1*} Naomi Sagan^{1*} Ryunosuke Goto^{2*} Jurik Mutter² Nick Phillips²
 Ash Alizadeh² Kangwook Lee⁴ Jose Blanchet¹ Mert Pilanci^{1†} Robert Tibshirani^{2,3‡}
¹Stanford University School of Engineering ²Stanford University School of Medicine
³Stanford University Department of Statistics ⁴University of Wisconsin-Madison
 {yz4232, nsagan, jose.blanchet, pilanci}@stanford.edu
 {rgoto, mutterj, nphil122, arasha, tibs}@stanford.edu
 {kangwook.lee}@wisc.edu

Abstract

We introduce LLM-Lasso, a novel framework that leverages large language models (LLMs) to guide feature selection in Lasso ℓ_1 regression. Unlike traditional methods that rely solely on numerical data, LLM-Lasso incorporates domain-specific knowledge extracted from natural language, optionally enhanced through a retrieval-augmented generation (RAG) pipeline, to seamlessly integrate data-driven modeling with contextual insights. Specifically, the LLM generates penalty factors for each feature, which are converted into weights for the Lasso penalty using a simple, tunable model. This is, to our knowledge, the first embedded LLM-driven feature selector. By design, LLM-Lasso addresses the key robustness challenges of LLM-driven feature selection: the risk of LLM hallucinations or low-quality responses. An internal cross-validation step is crucial to LLM-Lasso’s robustness, determining how heavily the prediction pipeline relies on the LLM’s outputs. Consequently, irrespective of the LLM’s generation quality, LLM-Lasso is guaranteed never to perform worse than standard Lasso. In various biomedical case studies, LLM-Lasso outperforms standard Lasso and existing feature selection baselines.

1 Introduction

Feature selection remains a cornerstone of statistical learning, enabling models to focus on the most relevant predictors while reducing complexity and improving interpretability [23, 8, 39]. Among the various methods for feature selection, Lasso regression has gained widespread adoption for various reasons. It delivers a feature selection approach while simultaneously building a predictive model. The Lasso approach is interpretable and computationally efficient because it automatically selects a suitable linear model with a sparse set of coefficients. Selection is performed by solving a straightforward convex optimization problem that promotes sparsity by penalizing the size of the regression coefficients [66, 7, 25]. As with any supervised learning model, the traditional Lasso approach is based only on the training data. It is natural to consider task-specific expert knowledge to inform the feature selection task. However, this is challenging to do in a systematic, scalable, and robust way. *We meet this challenge by augmenting Lasso with metadata extracted via generative AI.*

The development of large language models (LLMs) trained on a large scale of unstructured text offers a transformative opportunity to augment traditional feature selection techniques. Transformer-based pre-trained LLMs, such as GPT-4 [50] and LLaMA-2 [68] have demonstrated impressive abilities in encoding domain knowledge and contextual relationships and generalizing to a wide range of

unseen tasks in a variety of domains [70, 6, 57, 46], including various challenging reasoning tasks [71, 37, 64], prediction tasks that require domain-specific knowledge [55, 16, 9, 65, 13], and, more recently, feature selection [11, 29, 38, 42, 24]. [16] leverages LLM knowledge directly for regression problems by fine-tuning an LLM with training data, feature names, and task descriptions, showing comparable performance to traditional methods in low dimensions.

Efforts to incorporate LLMs into feature selectors have yielded filter and wrapper methods that perform competitively in low dimensions. [11] introduces the LMPriors framework, which selects features by analyzing log-probability differences when generating “Y” (Yes) or “N” tokens, admitting or rejecting certain features based solely on task descriptions, feature names, and few-shot examples. For proprietary LLMs where internal token probabilities are inaccessible, [29] proposes three prompting strategies that rely only on textual information. Such LLM-based filter feature selectors directly utilize the output of the generated text without further processing, making them sensitive to LLM inaccuracies. [42] provides a feature selection pipeline, starting with features selected by classical methods and using several epochs of LLM queries, informed by cross-validation accuracies for selected feature sets, to refine the features. As this is inherently wrapper-like, i.e., it doesn’t see the metadata and data together, multiple iterations of LLM queries are required.

These methods for incorporating LLMs into feature selection have shown promising results, demonstrating that LLMs can rival leading statistical techniques by encoding rich, task-specific knowledge. However, current approaches have yet to produce LLM-informed embedded feature selectors—methods that integrate feature metadata directly into regression models—and instead rely on multiple rounds of prompting or lack robustness compared to data-driven techniques. Most notably, existing methods make standalone feature selection decisions based solely on textual descriptions of the task and features, without incorporating mechanisms to safeguard against inaccuracies in LLM responses. This leaves them vulnerable to hallucinations—fabricated or inaccurate information—a well-documented weakness of even the most advanced LLMs [27, 75]. Such vulnerabilities raise concerns about reliability, especially in high-stakes domains like biomedicine, where data may be noisy or incomplete and precision is critical. *We address these limitations by introducing a robust framework for LLM-guided embedded feature selection, which integrates feature metadata directly into the learning algorithm and uses cross-validation to guard against poor-quality LLM responses.*

In this work, we introduce LLM-Lasso, a novel framework for LLM-powered feature selection that integrates LLM-derived penalty factors into Lasso penalty terms, allowing the seamless fusion of knowledge-based insights with traditional data-driven supervised learning methodologies. LLM-Lasso assumes black-box access to the LLMs and utilizes an optional retrieval-augmented generation (RAG) pipeline [36, 59, 73, 61] to extract domain-specific knowledge via LLMs, which is then used to inform Lasso regularization.

Main Contributions In this paper, we tackle the key bottleneck of robustness in task-specific modeling. We demonstrate the effectiveness of LLM-Lasso through extensive experiments, focusing on high-dimensional oncology tasks where the number of features exceeds those in prior studies by at least an order of magnitude. Our main contributions are as follows.

1. *We use contextual knowledge from LLMs to inform Lasso*, providing a scalable and reliable framework to directly integrate LLMs into traditional supervised learning methods. Experiments show that this task-specific information improves Lasso’s accuracy; LLM-Lasso consistently outperforms popular feature selection methods across various datasets.
2. *We ensure robustness by using the data to cross-validate LLM decisions*, using k -fold cross-validation to choose from a family of transformations on the LLM-generated penalty factors. This step ensures that LLM-Lasso can never perform worse than Lasso itself, a guarantee that LLM-based filter feature selectors lack.

The paper is structured as follows: Section 2 reviews Lasso, RAG, and presents a schematic of our procedure. Section 3 details our methodology. Section 4 determines penalty factor selection via simulations and exemplify robustness of LLM-Lasso through adversarial experiments. Section 5 evaluates LLM-Lasso across diverse datasets and LLMs. Finally, Section 6 summarizes our findings.

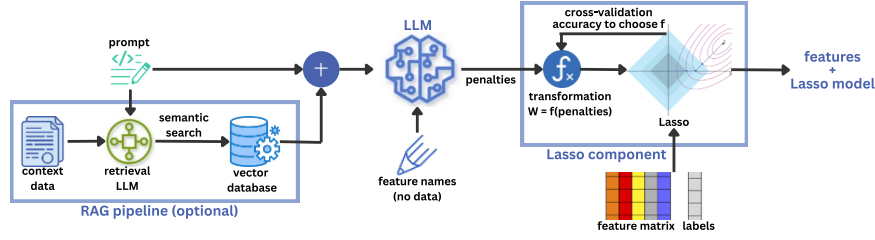


Figure 1: Full pipeline of collecting and using ℓ_1 -norm penalty factors in LLM-Lasso.

2 Preliminaries

2.1 Supervised Data-Driven Learning

We consider a generic data-driven supervised learning procedure. Given a dataset $\mathcal{D}$ consisting of n data points $(x_i, y_i) \in \mathcal{X} \times \mathcal{Y}$ drawn from an underlying distribution $p(\cdot|\theta)$, our goal is to estimate parameters $\theta \in \Theta$ through a learning procedure, defined as $f : (\mathcal{X} \times \mathcal{Y})^n \rightarrow \Theta$ that minimizes the predictive error on observed data. Specifically, the learning objective is defined as follows:

$$\hat{\theta}_f := f(\mathcal{D}) = \arg \min_{\theta} \mathcal{L}(\theta, \mathcal{D}), \quad (1)$$

where $\mathcal{L}(\cdot, \mathcal{D}) := \sum_{i=1}^n \mathcal{L}(\cdot, (x_i, y_i))$, and $\mathcal{L}$ is a loss function quantifying the error between predictions and true outcomes. Here, $\hat{\theta}_f$ is the parameter that best explains the observed data pairs (x_i, y_i) according to the chosen loss function $\mathcal{L}(\cdot)$.

Feature Selection. Feature selection aims to improve model f 's predictive performance while minimizing redundancy. State-of-the-art techniques fall into four categories: (i) filter methods, which rank features based on statistical properties like Fisher score [18, 62]; (ii) wrapper methods, which evaluate model performance on different feature subsets [32]; (iii) embedded methods, which integrate feature selection into the learning process using techniques like regularization [66, 34]; and (iv) hybrid methods, which combine elements of (i)-(iii) [60, 40]. This paper focuses on embedded methods via Lasso, benchmarking against approaches from (i)-(iii).

2.2 Language Modeling

Language modeling aims to approximate the true distribution of natural language $p_{\text{text}}(x)$ by learning $p_{\text{LM}}(x)$, a probability distribution over text sequences $x = (X_1, \dots, X_{|x|})$. Modern large language models, trained on diverse datasets [20], exhibit strong generalization across domains, acquire contextual knowledge, and perform zero-shot learning—solving new tasks using only task descriptions—or few-shot learning by leveraging a small number of demonstrations [6].

Retrieval-Augmented Generation (RAG). Retrieval-Augmented Generation (RAG) enhances the performance of generative language models by integrating a domain-specific information retrieval process [36]. The RAG framework comprises two main components: *retrieval*, which extracts relevant information from external knowledge sources, and *generation*, where an LLM generates context-aware responses using the prompt combined with the retrieved context. Documents are indexed through various databases, such as relational, graph, or vector databases [31, 17, 54], enabling efficient retrieval via algorithms like semantic similarity search to match the prompt with relevant documents in the knowledge base. RAG has gained much traction recently due to its demonstrated ability to reduce incidence of hallucinations and boost LLMs' reliability and performance [28, 76].

2.3 Task-Specific Data-Driven Learning

Task-specific data-driven learning augments information encoded in the data with *metadata*, or relevant domain expertise, to produce accurate, reliable, and interpretable models. For more information on model-specific feature relevance, see Appendix A.

LLM-Lasso aims to bridge the gap between data-driven supervised learning and the predictive capabilities of LLMs trained on rich metadata. This fusion not only enhances traditional data-driven methods by incorporating key task-relevant contextual information often overlooked by such models, but can also be especially valuable in low-data regimes, where the learning algorithm $f : \mathcal{D} \rightarrow \Theta$ (seen as a map from datasets $\mathcal{D}$ to the space of decisions Θ) is susceptible to overfitting.

The task-specific data-driven learning model $\tilde{f} : \mathcal{D} \times \mathcal{D}_{\text{meta}} \rightarrow \Theta$ can be described as a metadata-augmented version of f , where a link function $h(\cdot)$ integrates metadata (i.e. $\mathcal{D}_{\text{meta}}$) to refine the original learning process. This can be expressed as:

$$\tilde{f}(\mathcal{D}, \mathcal{D}_{\text{meta}}) := \mathcal{T}(f(\mathcal{D}), h(\mathcal{D}_{\text{meta}})),$$

where the functional $\mathcal{T}$ takes the original learning algorithm $f(\mathcal{D})$ and transforms it into a task-specific learning algorithm $\tilde{f}(\mathcal{D}, \mathcal{D}_{\text{meta}})$ by incorporating the metadata $\mathcal{D}_{\text{meta}}$.

There are multiple approaches to formulate $\mathcal{T}$ and h . For instance, LMPriors [11] designed h and $\mathcal{T}$ such that $h(\mathcal{D}_{\text{meta}})$ first specifies which features to retain (based on a probabilistic prior framework), and then $\mathcal{T}$ keeps the selected features and removes all the others from the original learning objective of f . Note that this approach inherently is restricted as it selects important features solely based on $\mathcal{D}_{\text{meta}}$ without seeing $\mathcal{D}$.

In contrast, we directly embed task-specific knowledge into the optimization landscape through regularization by introducing a structured inductive bias. This bias guides the learning process toward solutions that are consistent with metadata-informed insights, without relying on explicit probabilistic modeling. Abstractly, this can be expressed as:

$$\hat{\theta}_{\tilde{f}} := \tilde{f}(\mathcal{D}, \mathcal{D}_{\text{meta}}) = \arg \min_{\theta} \mathcal{L}(\theta, \mathcal{D}) + \lambda R(\theta, \mathcal{D}_{\text{meta}}), \quad (2)$$

where λ is a regularization parameter, $R(\cdot)$ is a regularizer, and θ is the prediction parameter.

3 Methodology

We introduce the LLM-Lasso framework, consisting of two main components: (i) a core statistical model that integrates LLM-informed penalties into Lasso; and (ii) a pipeline that incorporates expert knowledge into a task-specific LLM. While the RAG module in (ii) is optional, it can substantially improve accuracy for certain tasks. Figure 1 provides an overview.

3.1 The LLM-Lasso

We focus on the supervised learning framework introduced earlier in Section 2.1 with input feature $X \in \mathbb{R}^{n \times p}$ and response $Y \in \mathbb{R}^n$. The Lasso is a shrinkage method that places an ℓ_1 penalty on the coefficient, causing some coefficients to be exactly zero. The objective function of the Lasso is:

$$\min_{\beta} \left\{ \frac{1}{2} \sum_{i=1}^n (y_i - \beta_0 - x_i^\top \beta)^2 + \lambda \sum_{j=1}^p |\beta_j| \right\}. \quad (3)$$

To incorporate prior knowledge of the relationship between X and Y into the learning of a prediction model $f : (\mathcal{X} \times \mathcal{Y})^n \rightarrow \Theta$, one can enhance the Lasso by assigning penalty factors to each coefficient in the ℓ_1 penalty [77]. The objective function of the Lasso with penalty factors is:

$$\min_{\beta} \left\{ \frac{1}{2} \sum_{i=1}^n (y_i - \beta_0 - x_i^\top \beta)^2 + \lambda \sum_{j=1}^p w_j |\beta_j| \right\}. \quad (4)$$

While penalty factors can be manually assigned based on prior knowledge, this approach becomes impractical when p is large. Prior works such as [3, 69] seek to overcome this by exploiting external datasets or citation-based weighting to compute penalty factors. We take a similar but potentially more comprehensive approach: we leverage LLMs to streamline the integration of task-specific knowledge by generating LLM-informed penalty factors or importance scores for all predictors using domain-specific insights. The key modeling challenge is determining how best to effectively inform the underlying data-driven shrinkage method. In the following, we introduce a framework for modeling LLM-informed penalty factors.

3.1.1 The Cross-Validation Procedure

In addition to incorporating LLM penalties into the Lasso, we develop a data-driven cross-validation procedure to tune the model’s reliance on LLM knowledge.

Given the LLM-generated penalty factors $V \in \mathbb{R}^p$, we compute the final LLM-Lasso penalties $W^* \in \mathbb{R}^p$ by performing a transformation $\tau^* : \mathbb{R}^p \rightarrow \mathbb{R}^p$ on V . τ^* is chosen, using cross-validation, from a finite family of transformations, $\mathcal{T}$.

Via k -fold cross validation, we obtain k different training and validation sets of the data. For split i , let $\beta_{i,\tau(V)}^*$ be the optimal Lasso coefficients given the training data from the i^{th} split and penalties $W = \tau(V)$. Let $X_{\text{val}}^{(i)}$ and $y_{\text{val}}^{(i)}$ refer to the validation data and labels for that split. Denoting the cross-validation loss function by $\mathcal{L}$, the final LLM penalties are

$$W^* = \tau^*(V), \quad \text{where} \quad \tau^* = \arg \min_{\tau \in \mathcal{T}} \sum_{i=1}^k \mathcal{L} \left(X_{\text{val}}^{(i)} \beta_{i,\tau(V)}^*, y_{\text{val}}^{(i)} \right).$$

We define $\mathcal{T}$ to have a range of transformations representing different degrees of reliance on LLM-generated penalties. If the LLM penalties are high-quality, the cross-validation procedure chooses τ^* to accentuate the LLM penalties, i.e., increase the relative distance between low penalties and high penalties. On the other hand, if $\mathcal{T}$ includes the transformation $\tau_0(W) = 1$ (i.e., one that maps any penalties to those of the plain Lasso), then LLM-Lasso can never do worse than plain Lasso in cross-validation loss.

Choosing a Transformation Family One such family of transformations is known as the *inverse importance family*,¹

$$\mathcal{T} = \left\{ \tau : \tau(V)_j = v_j^\eta, \eta \in \{0, 1, \dots, \eta_{\max}\} \right\}.$$

For $\eta = 0$, the resulting penalties are the same as plain Lasso, indicating no reliance on the LLM output. Likewise, a high value of η indicates heavy reliance on the LLM scores.

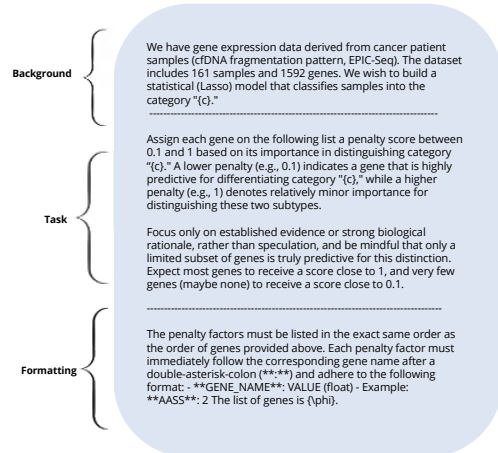
In addition to inverse importance penalty factors, we also consider a ReLU-based family of penalty transformations. We run simulations to find the better form of penalty factors. Based on the simulations, we use the inverse importance penalty factors to compare the LLM-Lasso to the baseline models. We defer the details of the ReLU transformation family and simulations to Appendix B.

3.2 Task-Specific LLM

To develop a task-specific LLM that provides accurate answers grounded in rigorous domain knowledge, we focus on two key aspects: prompt engineering and knowledge-base embedding via RAG.

3.2.1 Prompt Engineering.

Prompting is an efficient and effective approach for adapting pretrained LLMs to tackle new tasks not encountered during training [57, 41]. In our experiment, we employ a zero-shot approach for large-scale experiment on biomedical dataset, where the acquisition of ground truth is often infeasible, and a few-shot approach for small-scale experiments. By default, we use greedy decoding—i.e., sampling with temperature $T = 0$ —due to its simplicity and deterministic behavior, making it well-suited for replication and ablation studies. In addition, we incorporate chain-of-thought (CoT) prompting [71], a technique shown to significantly enhance performance on complex reasoning tasks.



¹The LLM-generated penalties can be viewed as $v_j = \frac{1}{L_j}$, where L_j is the importance of feature j .

For all classification tasks, our full prompt template consists of three components—user, retriever (if RAG is used), and system—and is defined as follows:

$$\mathcal{P}^{\text{full}} = \text{prompt}(\mathcal{Q}^{\text{user}}(\mathcal{A}(\phi, c)), \mathcal{C}^{\text{retriever}}(k, \mathcal{R}(\phi, c)), \mathcal{H}^{\text{system}}),$$

where (i). Q^{user} stands for user query, which is comprised of $\mathcal{A}$, a task description prompt that takes features ϕ and categories c as inputs; (ii). $\mathcal{C}^{\text{retriever}}$ represents the top k retrieved contexts via a semantic similarity search of retrieval prompt $\mathcal{R}(\phi, c)$ with the retrieval knowledge base; and (iii). $\mathcal{H}^{\text{system}}$ summarizes past queries and responses, enacted through a conversational buffer. Under this framework, prompt engineering consists of three components: $\mathcal{A}$ (task description), and $\mathcal{R}$ (retrieval prompt). Component $\mathcal{A}$ follows the general structure in Figure 2, where it is composed of a background description of the dataset, the assigned the task, and formatting instructions. We refer the readers to Appendix C for a more detailed description.

3.2.2 Knowledge Base Embedding via RAG

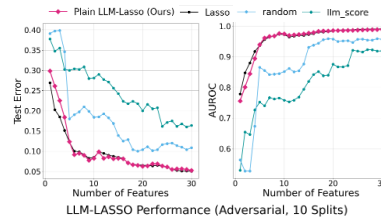
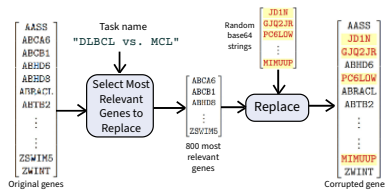
We use a standard RAG pipeline to optionally embed a task-specific knowledge base for our prediction task. RAG augments LLMs with relevant contextual information by retrieving documents from a database—crucial given LLMs’ limitations with long contexts and large inputs. Below, we outline the specific RAG pipeline used.

Preprocessing Given a knowledge base of N text documents, $\{D_i\}_{i=1}^N$, we obtain their d -dimensional semantic embeddings $\{d_i\}_{i=1}^N = \{E(D_i)\}_{i=1}^N$ via an embedding function $E : \text{Text} \rightarrow \mathbb{R}^d$. Here, we use the OpenAI embeddings off-the-shelf [51]. Upon obtaining the semantic embedding vectors we apply the Hierarchical Navigable Small World (HNSW) algorithm [45], implemented in chromadb, to enable sublinear complexity for semantic similarity search.

Retrieval At retrieval time, given a query vector $q \in \mathbb{R}^d$, the semantic similarity between q and the stored embeddings $\{d_i\}_{i=1}^N$ is computed as $\text{Sim}(q, d_i) = \frac{q^T d_i}{\|q\|_2 \|d_i\|_2}$. The top k documents with the highest similarity scores are retrieved and supplied as context $\mathcal{C}$.

Throughout the paper, we adhere to the following naming convention: LLM-Lasso (Plain) refers to a pipeline without RAG, while LLM-Lasso (RAG) denotes a pipeline incorporating RAG. The performance of RAG in our framework highly depends on the retrieval prompt and the relevance of the retrieved documents. Figure 1 illustrates LLM-Lasso (RAG). Due to space constraints, a detailed discussion is provided in Appendix E.

4 Adversarial Simulations



(a) Gene name corruption for adversarial simulations. (b) Adversarial simulation results: DLBCL vs. MCL.

Figure 3: Adversarial simulation diagram (left) and results (right).

To showcase the robustness of our method in scenarios in which the LLM fails to produce meaningful results, we perform adversarial data corruption simulations. As a base dataset, we use the myeloid cell leukemia (MCL) vs. diffuse large B-cell lymphoma (DLBCL) task from the Lymphoma dataset (Table 5). Of the 1592 gene features, we select the 800 most relevant based on presence in documents retrieved from the OMIM (Online Mendelian Inheritance in Man) knowledge base (see Section 5.4.2).

We replace those genes with random base64 strings, ensuring via OMIM that the strings are not real gene names (see Figure 3a).

We perform classification via LLM-Lasso and LLM-Score, as described in Section 5, using the GPT-4o model. Both methods are given the corrupted gene name list. For illustrative purposes, we also include a random feature selection baseline. The resulting misclassification error and AUROC plots can be found in Figure 8. We observe that for both LLM-Lasso and LLM-Score, the LLM analysis of corrupted genes is heavily based on hallucinations, examples of which are in Figure 13 in the Appendix. Even so, the accuracy of LLM-Lasso remains comparable to Lasso, whereas LLM-Score performs noticeably worse than random feature selection.

5 Experiments

In this section, we demonstrate the effectiveness of our proposed framework, LLM-Lasso, through a series of experiments. These include small-scale tests (~ 20 features) and large-scale biomedical experiments (> 1000 features). Via these experiments, we demonstrate the following:

1. *Metadata improves Lasso performance*: as is evident in the large-scale experiments, the use of LLM-generated penalty factors consistently boosts the performance of Lasso (Figure 6).
2. *LLM-Lasso is much more robust than comparable LLM-driven methods*: though LLM-Score and LMPriors perform well in a low-dimensional settings, LLM-Score performance degrades for the high-dimensional biomedical examples (and LMPriors becomes cost-prohibitive), whereas LLM-Lasso shines in a high-dimensional setting (Figure 5).
3. *LLM-Lasso is competitive in a broad variety of settings*: LLM-Lasso performance matches, often outperforms, baselines in both large- and small-scale experiments (Figures 5 and 4).

Refer to Appendix D.3 for additional experimental results, including an analysis of the relevance of features selected by LLM-Lasso in the large-scale experiments.

5.1 Model Details

For the experiment, we sample a combination of closed-source and open-source LLMs, as described in Table 1. We use all GPT models via OpenAI API calling and all open-source models via OpenRouter API calling via cloud-based inference. We implement RAG using the langchain-community [33] code-base and a self-query retriever as our base method for query construction via Chroma vectorstore.

Table 1: LLMs used in LLM-Lasso experiments.

Model	o1 [52]	GPT-4o [50]	GPT-3.5 [49]	DeepSeek-R1 [14]	LlaMa-3.1 [48]	LlaMA-3 [47]	Qwen [1]
Parameters	~ 2	—	—	671B	405B	8B	72B

5.2 Baselines

To robustly evaluate our model’s performance, we compare it against baselines from both LLM-based feature selectors and traditional data-driven feature selection methods, with representatives chosen from each of the three main categories, that is, filter, wrapper, and embedded: (1) LLM-Score [29], (2) LMPriors [11]³, (3) Filtering by Mutual Information (MI) [35], (4) Recursive Feature Elimination (RFE) [22], (5) Minimum Redundancy Maximum Relevance selection (MRMR) [15], (6) Lasso [66], (7) XGBoost [10], (8) Random feature selection.

For the standalone feature selectors LLM-Select, MI, RFE, MRMR, and Random, we follow the procedures outlined in [29] to ensure a fair comparison:

³We only consider the LMPriors baseline for the small-scale experiments, as it is cost-prohibitive to perform one API query per feature for high-dimensional problems.

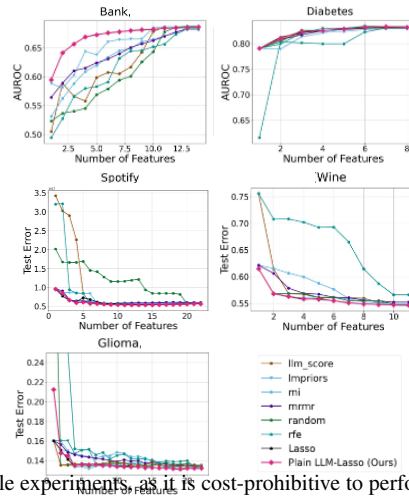


Figure 4: Small-scale experiments on public datasets using GPT-4o.

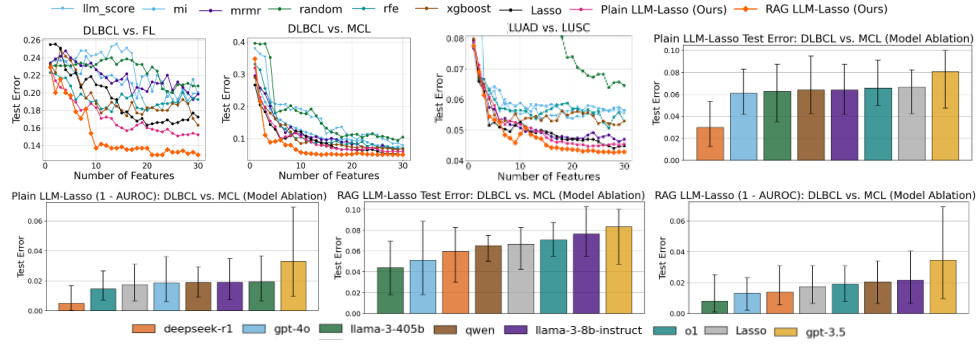


Figure 5: Large-Scale Experiments: LLM-Lasso vs. Baselines and LLM Model Ablation. The model ablations show mean test misclassification and $1 - \text{AUROC}$ at 20 features. Error bars are 0.95 and 0.05 quantiles over 10 training and test splits, computed via the `pandas` `quantile` function.

their performance is evaluated by measuring the test performance of a downstream ℓ_2 -penalized logistic regression model, with hyperparameters chosen via grid search and cross-validation.

5.3 Small-Scale Experiments

We begin with a preliminary evaluation of LLM-Lasso against baselines using small-scale, low-dimensional public datasets across various domains. This includes three binary classification datasets (Bank, Diabetes, Glioma) and two regression datasets (Wine Quality, Spotify 2024). Spotify was published after the pretraining-data cutoff for all sampled LLMs (see Table 3), included to mitigate concerns about pretraining-data memorization. A summary of the datasets used can be found in Table 4. We follow the evaluation procedures outlined in Section 5.2 for standalone feature selectors and in Appendix E.3 for Lasso-based models. To ensure a fair assessment in the presence of class imbalance, we report the error rate across ten splits along with the AUROC. As shown in Figure 4, GPT-4o-based LLM-Lasso consistently outperforms all sampled datasets and baselines, even when not using the best-performing LLM (see Appendix D.3.3 for a model ablation study on Spotify).

5.4 Large-Scale Experiments

Gene expression levels support cancer diagnosis and prediction, while identifying predictive genes deepens our understanding of cancer and aids drug discovery. To showcase the applicability and strong performance of LLM-Lasso on high-dimensional, complex data, we evaluate it on cancer diagnosis and classification tasks using gene expression features across diverse biomedical settings.

5.4.1 Datasets

To address concerns on LLM memorization and ensure transparency and reproducibility, we conduct experiments on both an unpublished lymphoma dataset and a publicly available lung cancer dataset.

Lymphoma (Unpublished) Follicular lymphoma (FL) is a relatively indolent form of lymphoma that usually does not require intervention, but it could occasionally transform into the more aggressive diffuse large B-cell lymphoma (DLBCL). Using an unpublished dataset of 1592 gene expression levels from 130 lymphoma samples, we use LLM-Lasso to classify tumor samples into DLBCL and FL. Though less clinically significant, we also perform the task of classifying 161 lymphoma samples into DLBCL and mantle cell lymphoma (MCL) using 1592 gene expression levels. The datasets used are summarized in Table 5. More information about these tasks can be found in Appendix D.2.

Lung Cancer (Public) We additionally perform evaluations on an open-source lung cancer dataset. Data are obtained from The Cancer Genome Atlas Program (TCGA) [72], a publicly available database of human tumors. Our sample consists of bulk RNA sequencing data from 516 samples

from patients with lung adenocarcinoma (LUAD) and 501 samples from patients with lung squamous cell carcinoma (LUSC). Only primary tumor samples are used. The data are normalized, variance-stabilized, and transformed using DESeq2 [44]. If more than one sequencing data is available for a patient, the average of the counts are taken. Genes with fewer than 10 counts are filtered out. We use the top 1000 most variable protein-coding genes in the downstream analyses.

5.4.2 Building a Knowledge-Base for RAG

We use OMIM (Online Mendelian Inheritance in Man), an open-source database of human genes and associated diseases, to construct our RAG knowledge base. Gene symbols, titles, clinical synopses, and genetic-phenotypic relationships are extracted via the OMIM API and stored in structured JSON. This data is then chunked with a recursive text splitter and indexed into a Chroma vector store.

5.4.3 Evaluation

Evaluation of Prediction Performance Results from large-scale experiments are shown in Figure 5. Figure 6 shows the “win ratio” between LLM-Lasso and Lasso, i.e., the ratio of points in the first 30 features where RAG LLM-Lasso is strictly better than Lasso, and vice versa. RAG LLM-Lasso outperforms both the baselines and plain LLM-Lasso, achieving lower misclassification rates with fewer selected genes. Refer to Appendix E.3 for evaluation methodology, and Appendix D.3.3 for AUROC plots and a close-up comparison of RAG LLM-Lasso vs. Lasso.

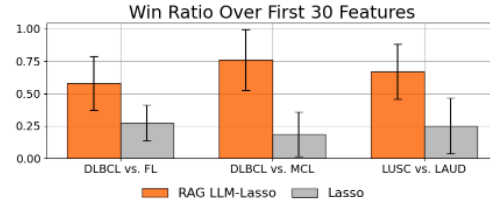


Figure 6: Win ratio between LLM-Lasso and Lasso, averaged over 10 train/test splits with standard deviation error bars⁴ computed via `pandas.DataFrame.agg`.

Evaluation of LLM performance The histograms of Figure 5 display the average misclassification error and AUROC of LLM-Lasso at 20 features, for the models listed in Section 5.1. Lasso is plotted as a baseline. Larger and more powerful models generally perform better, especially with RAG. Some key exceptions are o1, which does worse than the smaller GPT-4o model, and DeepSeek-R1, for which RAG degrades performance. We hypothesize that some models have more nuanced abilities to parse the medical documents provided by RAG, whereas others are harmed by the increased context from the retrieved documents.

Evaluation of RAG performance. We evaluate RAG via (i) the recall@k metric, and (ii) impact of RAG on LLM-Lasso accuracy. We evaluate recall for the OMIM vector database from Section 5.4.2. We first construct 100 retrieval queries by randomly sampling without replacement from the retrieval queries performed during the lymphoma dataset experiments. Then, we compare ground truth nearest documents (by cosine similarity) for each query to those retrieved via Chroma’s query interface. Results of this experiments are in Figure 7. The recall numbers appear reasonable for our purposes; the vector database is consistently retrieving the documents with the closest embeddings, even if not all of the time. Discussion on the impact of RAG on LLM accuracy can be found in Appendix D.3.4.

k	1	3	5	10	25	50
Recall (%)	90.74	87.04	86.30	85.19	82.52	79.24

Figure 7: Recall@k for the OMIM RAG system

6 Discussion and Conclusion

The LLM-Lasso is a simple, tunable model that incorporates domain-specific knowledge from LLMs and outperforms state-of-the-art feature selection models. It achieves strong performance with a small number of features, improving both predictive accuracy and interpretability by highlighting informative variables. LLM-Lasso safeguards against potential inaccuracies or hallucinations from the LLM through hyperparameter tuning, as demonstrated in the FL experiment. The cross-validated inverse importance transformation further allows the model to modulate its reliance on LLM-derived penalty factors based on data-driven validation.

⁴Such error bars implicitly assume normally-distributed win ratios.

While LLM-Lasso is scalable and task-adaptable, it has several limitations that present promising directions for future work. First, the current querying costs of the LLM component could be further optimized. Second, although cross-validation helps down-weight unreliable LLM scores, the LLM itself remains static; incorporating feedback from validation outcomes, e.g., via prompt tuning or fine-tuning, could improve adaptability and robustness. Third, key design components such as the choice of transformation family, prompting strategy, and the construction of the RAG database warrant deeper empirical study to better understand their impact on performance and generalization.

References

- [1] Alibaba DAMO Academy. Qwen models, 2025. URL <https://damo.alibaba.com/qwen>. Foundation models developed by Alibaba DAMO Academy.
- [2] Sietse M Aukema, Roel van Pel, Inga Nagel, Susanne Bens, Reiner Siebert, Stefano Rosati, Eva van den Berg, Anneke G Bosga-Bouwer, Robby E Kibbelaar, Mels Hoogendoorn, et al. Myc expression and translocation analyses in low-grade and transformed follicular lymphoma. *Histopathology*, 71(6):960–971, 2017.
- [3] Linn Cecilie Bergersen, Ingrid K Glad, and Heidi Lyng. Weighted lasso with data integration. *Statistical applications in genetics and molecular biology*, 10(1), 2011.
- [4] Leo Breiman. Random forests. *Machine learning*, 45:5–32, 2001.
- [5] Leo Breiman. Manual on setting up, using, and understanding random forests v3. 1. *Statistics Department University of California Berkeley, CA, USA*, 1(58):3–42, 2002.
- [6] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language Models are Few-Shot Learners. *arXiv preprint arXiv:2005.14165*, 2020. URL <https://arxiv.org/abs/2005.14165>.
- [7] Peter Bühlmann and Sara Van De Geer. Statistics for high-dimensional data: Methods, theory and applications. *Springer*, 2011.
- [8] Girish Chandrashekar and Ferat Sahin. A survey on feature selection methods. *Computers & Electrical Engineering*, 40(1):16–28, 2014.
- [9] Jing Chen and Xiaoming Zou. Prediction tasks using embeddings derived from domain-specific literature with large language models. *Advances in Data Science and Analytics*, 8(2):200–215, 2024.
- [10] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*, pages 785–794, 2016.
- [11] Kristy Choi, Chris Cundy, Sanjari Srivastava, and Stefano Ermon. Lmpriors: Pre-trained language models as task-specific priors, 2022. URL <https://arxiv.org/abs/2210.12530>.
- [12] Cristina Correia, Paula A Schneider, Haiming Dai, Ahmet Dogan, Matthew J Maurer, Amy K Church, Anne J Novak, Andrew L Feldman, Xiaosheng Wu, Husheng Ding, et al. Bcl2 mutations are associated with increased risk of transformation and shortened survival in follicular lymphoma. *Blood, The Journal of the American Society of Hematology*, 125(4):658–667, 2015.
- [13] Haoyang Cui, Chen Wang, Haroon Maan, and Bin Wang. scgpt: Towards building a foundation model for single-cell multi-omics using generative ai. *Nature Methods*, 2024.
- [14] DeepSeek AI. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning, 2025.
- [15] Chris Ding and Hanchuan Peng. Minimum redundancy feature selection from microarray gene expression data. *Journal of Bioinformatics and Computational Biology*, 3(02):185–205, 2005.
- [16] Tuan Dinh, Yuchen Zeng, Ruisu Zhang, Ziqian Lin, Michael Gira, Shashank Rajput, Jy yong Sohn, Dimitris Papailiopoulos, and Kangwook Lee. LIFT: Language-interfaced fine-tuning for non-language machine learning tasks. In Alice H. Oh, Alekh Agarwal, Danielle Belgrave, and Kyunghyun Cho, editors, *Advances in Neural Information Processing Systems*, 2022. URL https://openreview.net/forum?id=s_PJMEGIUfa.

- [17] Matthijs Douze, Andrey Guzhva, Cheng Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Manuel Lomeli, Lida Hosseini, and Hervé Jégou. The faiss library, 2024. Available at <https://faiss.ai>.
- [18] Richard O. Duda, Peter E. Hart, and David G. Stork. *Pattern Classification*. John Wiley & Sons, New York, 2 edition, 2001.
- [19] Jerome Friedman, Robert Tibshirani, and Trevor Hastie. Regularization paths for generalized linear models via coordinate descent. *Journal of Statistical Software*, 33(1):1–22, 2010. doi: 10.18637/jss.v033.i01.
- [20] Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, Shawn Presser, and Connor Leahy. The pile: An 800gb dataset of diverse text for language modeling. *arXiv preprint arXiv:2101.00027*, December 2020. URL <https://arxiv.org/abs/2101.00027>.
- [21] Michael R Green, Andrew J Gentles, Ramesh V Nair, Jonathan M Irish, Shingo Kihira, Chih Long Liu, Itai Kela, Erik S Hopmans, June H Myklebust, Hanlee Ji, et al. Hierarchy in somatic mutations arising during genomic evolution and progression of follicular lymphoma. *Blood, The Journal of the American Society of Hematology*, 121(9):1604–1611, 2013.
- [22] Isabelle Guyon, Jason Weston, Stephen Barnhill, and Vladimir Vapnik. Gene selection for cancer classification using support vector machines. *Machine Learning*, 46:389–422, 2002.
- [23] Isabelle Guyon, Steve Gunn, Masoud Nikravesh, and Lotfi A Zadeh. *Feature Extraction: Foundations and Applications*. Springer, 2007.
- [24] Sungwon Han, Jinsung Yoon, Sercan O Arik, and Tomas Pfister. Large language models can automatically engineer features for few-shot tabular learning, 2024. URL <https://arxiv.org/abs/2404.09491>.
- [25] Trevor Hastie, Robert Tibshirani, and Martin Wainwright. *Statistical learning with sparsity: the lasso and generalizations*. Chapman and Hall/CRC, 2015.
- [26] Arthur E Hoerl and Robert W Kennard. Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1):55–67, 1970.
- [27] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, and Ting Liu. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Transactions on Information Systems*, 2024. ISSN 1558-2868. doi: 10.1145/3703155. URL <http://dx.doi.org/10.1145/3703155>.
- [28] Lifu Huang, Wei Yu, Weiyang Ma, Wenhan Zhong, Zihan Feng, Haoxue Wang, Qiang Chen, Wei Peng, Xinyu Feng, Bing Qin, and Ting Liu. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *arXiv preprint*, arXiv:2311.05232, November 2023. doi: 10.48550/arXiv.2311.05232. URL <https://doi.org/10.48550/arXiv.2311.05232>.
- [29] Daniel P. Jeong, Zachary C. Lipton, and Pradeep Ravikumar. Llm-select: Feature selection with large language models, 2024. URL <https://arxiv.org/abs/2407.02694>.
- [30] Javed Khan, Jun S Wei, Markus Ringner, Lao H Saal, Marc Ladanyi, Frank Westermann, Frank Berthold, Manfred Schwab, Cristina R Antonescu, Carsten Peterson, et al. Classification and diagnostic prediction of cancers using gene expression profiling and artificial neural networks. *Nature medicine*, 7(6):673–679, 2001.
- [31] Omar Khattab and Matei Zaharia. Colbert: Efficient and effective passage search via contextualized late interaction over bert. *arXiv preprint*, arXiv:2004.12832, April 2020. doi: 10.48550/arXiv.2004.12832. URL <https://doi.org/10.48550/arXiv.2004.12832>.
- [32] Ron Kohavi and George H. John. Wrappers for feature subset selection. *Artificial Intelligence*, 97(1):273–324, 1997.

- [33] LangChain Community. Langchain community resources, 2024. URL <https://docs.langchain.com>.
- [34] Ismael Lemhadri, Feng Ruan, Louis Abraham, and Robert Tibshirani. LassoNet: A neural network with feature sparsity. *Journal of Machine Learning Research*, 22(127):1–29, 2021.
- [35] D. D. Lewis. Feature selection and feature extraction for text categorization. In *Speech and Natural Language: Proceedings of a Workshop Held at Harriman, New York, February 23-26, 1992*, 1992.
- [36] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. *arXiv preprint arXiv:2005.11401*, 2020.
- [37] Aitor Lewkowycz, Anders Johan Andreassen, David Dohan, Ethan Dyer, Henryk Michalewski, Vinay Venkatesh Ramasesh, Ambrose Slone, Cem Anil, Imanol Schlag, Theo Gutman-Solo, Yuhuai Wu, Behnam Neyshabur, Guy Gur-Ari, and Vedant Misra. Solving quantitative reasoning problems with language models. In *Advances in Neural Information Processing Systems*, 2022.
- [38] Dawei Li, Zhen Tan, and Huan Liu. Exploring large language models for feature selection: A data-centric perspective, 2024. URL <https://arxiv.org/abs/2408.12025>.
- [39] Jundong Li and Huan Liu. Feature selection: An ever-evolving frontier in statistical learning. *IEEE Transactions on Neural Networks and Learning Systems*, 26(1):1–14, 2015.
- [40] Xinqian Li and Jia Ren. Micq-ipso: An effective two-stage hybrid feature selection algorithm for high-dimensional data. *Neurocomputing*, 501:328–342, Aug 2022. doi: 10.1016/j.neucom.2022.05.048. URL <https://doi.org/10.1016/j.neucom.2022.05.048>.
- [41] Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *ACM Computing Surveys*, 55(9), 2023.
- [42] S. Liu, F. Lv, X. Liu, et al. Ice-search: A language model-driven feature selection approach. *arXiv preprint*, arXiv:2402.18609, 2024. <https://arxiv.org/abs/2402.18609>.
- [43] IS Lossos, R Levy, and AA Alizadeh. Aid is expressed in germinal center b-cell-like and activated b-cell-like diffuse large-cell lymphomas and is not correlated with intracolon heterogeneity. *Leukemia*, 18(11):1775–1779, 2004.
- [44] Michael I Love, Wolfgang Huber, and Simon Anders. Moderated estimation of fold change and dispersion for rna-seq data with deseq2. *Genome biology*, 15:1–21, 2014.
- [45] Yu. A. Malkov and Dmitry A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2018. doi: 10.1109/TPAMI.2018.2889473.
- [46] Hariharan Manikandan, Yiding Jiang, and J. Zico Kolter. Language models are weak learners. *arXiv preprint arXiv:2306.14101*, 2023.
- [47] Meta AI. Llama 3 - 8b instruct model. <https://huggingface.co/meta-llama/llama-3-8b-instruct>, 2025. Accessed: 2025-01-16.
- [48] Meta AI. Llama 405b, 2025. URL <https://ai.meta.com/llama>. Large-scale language model with 405 billion parameters.
- [49] OpenAI. Gpt-3.5 technical report, 2023. URL <https://openai.com/research/gpt-3-5>. Accessed: YYYY-MM-DD.
- [50] OpenAI. Gpt-4 technical report, 2023. URL <https://openai.com/research/gpt-4>.
- [51] OpenAI. Openai embeddings, 2024. URL <https://platform.openai.com/docs/guides/embeddings>.

- [52] OpenAI. Openai o1 system card, 2024. URL <https://cdn.openai.com/o1-system-card.pdf>. Large language model.
- [53] Laura Pasqualucci and Riccardo Dalla-Favera. Genetics of diffuse large b-cell lymphoma. *Blood, The Journal of the American Society of Hematology*, 131(21):2307–2319, 2018.
- [54] Boci Peng, Yun Zhu, Yongchao Liu, Xiaohe Bo, Haizhou Shi, Chuntao Hong, Yan Zhang, and Siliang Tang. Graph retrieval-augmented generation: A survey, 2024. URL <https://arxiv.org/abs/2408.08921>.
- [55] Fabio Petroni, Tim Rocktäschel, Sebastian Riedel, Patrick Lewis, Anton Bakhtin, Yuxiang Wu, and Alexander Miller. Language models as knowledge bases? In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 2463–2473. Association for Computational Linguistics, 2019.
- [56] R Core Team. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria, 2024. URL <https://www.R-project.org/>.
- [57] Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. OpenAI Blog, 2019.
- [58] Sridhar Ramaswamy, Pablo Tamayo, Ryan Rifkin, Sayan Mukherjee, Chen-Hsiang Yeang, Michael Angelo, Christine Ladd, Michael Reich, Eva Latulippe, Jill P Mesirov, et al. Multiclass cancer diagnosis using tumor gene expression signatures. *Proceedings of the National Academy of Sciences*, 98(26):15149–15154, 2001.
- [59] Kurt Shuster, Samuel Humeau, Masoud Komeili, and Jason Weston. Improving fact-checking with retrieval-augmented generation. In *Proceedings of the 2022 Annual Meeting of the Association for Computational Linguistics (ACL)*, 2022.
- [60] Namrata Singh and Pradeep Singh. A hybrid ensemble-filter wrapper feature selection approach for medical data classification. *Chemometrics and Intelligent Laboratory Systems*, 217:104396, 2021. ISSN 0169-7439. doi: <https://doi.org/10.1016/j.chemolab.2021.104396>. URL <https://www.sciencedirect.com/science/article/pii/S0169743921001647>.
- [61] Shamane Siriwardhana, Rivindu Weerasekera, Elliott Wen, Tharindu Kaluarachchi, Rajib Rana, and Suranga Nanayakkara. Improving the domain adaptation of retrieval augmented generation (RAG) models for open domain question answering. *Transactions of the Association for Computational Linguistics*, 11:1–17, 2023. doi: 10.1162/tacl_a_00530. URL <https://aclanthology.org/2023.tacl-1.1/>.
- [62] Le Song, Alex Smola, Arthur Gretton, Justin Bedo, and Karsten Borgwardt. Feature selection via dependence maximization. *Journal of Machine Learning Research*, 13(47):1393–1434, 2012.
- [63] Avrum Spira, Jennifer E Beane, Vishal Shah, Katrina Steiling, Gang Liu, Frank Schembri, Sean Gilman, Yves-Martine Dumas, Paul Calner, Paola Sebastiani, et al. Airway epithelial gene expression in the diagnostic evaluation of smokers with suspect lung cancer. *Nature medicine*, 13(3):361–366, 2007.
- [64] Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc Le, Ed Chi, Denny Zhou, and Jason Wei. Challenging big-bench tasks and whether chain-of-thought can solve them. In *Findings of the Association for Computational Linguistics*, 2023.
- [65] Christina V. Theodoris, Ling Xiao, Aditya Chopra, Mark D. Chaffin, Zainab R. Al Sayed, Matthew C. Hill, Hannah Mantineo, Emily M. Brydon, Zheng Zeng, Xiaoli S. Liu, and Patrick T. Ellinor. Transfer learning enables predictions in network biology. *Nature*, 618(7965):616–624, 2023.
- [66] Robert Tibshirani. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society: Series B (Methodological)*, 58(1):267–288, 1996.

- [67] Robert Tibshirani. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 58(1):267–288, 1996.
- [68] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Marc’Aurelio Ranzato, Alexina S. A. Roux, Punit Singh Koura, Kristina Gong, Baptiste Rozière, David Belgrave, Mohamed El Hoseiny, Parsa Sakhaei, Mohammad Babaeizadeh, Spyridon Bakas, Diego de Las Casas, Tao Xu, Romain Larcher, Timothée Lacroix, Guillaume Lample, and Alexis Conneau. LLaMA 2: Open Foundation and Fine-Tuned Chat Models, 2023. URL <https://arxiv.org/abs/2307.09288>. Accessed: YYYY-MM-DD.
- [69] Daniel Urda, Francisco Aragón, Rocío Bautista, Francisco J. López, and José M. Pérez. BLASSO: integration of biological knowledge into a regularized linear model. *BMC Systems Biology*, 12(Suppl 5):94, 2018. doi: 10.1186/s12918-018-0612-8. URL <https://doi.org/10.1186/s12918-018-0612-8>.
- [70] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- [71] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, 2022.
- [72] John N Weinstein, Eric A Collisson, Gordon B Mills, Kenna R Shaw, Brad A Ozenberger, Kyle Ellrott, Ilya Shmulevich, Chris Sander, and Joshua M Stuart. The cancer genome atlas pan-cancer analysis project. *Nature genetics*, 45(10):1113–1120, 2013.
- [73] Shangyu Wu, Ying Xiong, Yufei Cui, Haolun Wu, Can Chen, Ye Yuan, Lianming Huang, Xue Liu, Tei-Wei Kuo, Nan Guan, and Chun Jason Xue. Retrieval-augmented generation for natural language processing: A survey, 2024. URL <https://arxiv.org/abs/2407.13193>.
- [74] James Yang and Trevor Hastie. A fast and scalable pathwise-solver for group lasso and elastic net penalized regression via block-coordinate descent, 2024. URL <https://arxiv.org/abs/2405.08631>.
- [75] Jia-Yu Yao, Kun-Peng Ning, Zhen-Hui Liu, Mu-Nan Ning, Yu-Yang Liu, and Li Yuan. Llm lies: Hallucinations are not bugs, but features as adversarial examples, 2024. URL <https://arxiv.org/abs/2310.01469>.
- [76] Yuchen Zhang, Mohammad Khalifa, Lajanugen Logeswaran, Mingu Lee, Hwaran Lee, and Lajanugen Wang. Merging generated and retrieved knowledge for open-domain qa. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 4710–4728, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.286. URL <https://aclanthology.org/2023.emnlp-main.286>.
- [77] Hui Zou. The adaptive lasso and its oracle properties. *Journal of the American statistical association*, 101(476):1418–1429, 2006.
- [78] Hui Zou and Trevor Hastie. Regularization and variable selection via the elastic net. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 67(2):301–320, 2005.

A Model-Specific Feature Relevance

Feature importance can be extracted directly from statistical models. For instance, the magnitude of coefficients in Lasso [67], ridge [26], and elastic net [78] regressions can be directly interpreted as feature importance, given that the features are standardized. Feature importance of tree-based methods, in general, is based on the improvements in accuracy brought by splits on a specific feature. For instance, XGBoost uses a "gain" metric to quantify the improvement in accuracy from a split [10], while random forests use mean decrease impurity, which measures the total reduction in impurity by all splits on a given feature [5]. Use of random permutations of features to evaluate feature importance [4] is also popular.

B Choosing a Transformation Family

B.1 ReLU-Form Penalty Factors

In addition to inverse importance penalties, another approach to defining the penalty factors involves interpolating between LLM-generated penalty factors and equal ℓ_1 -norm weights by applying a rectified linear unit (ReLU) function. Suppose τ is a transformation in the ReLU family. Let $V_{(j)}$ be the j^{th} -largest element of V , and $\tau(V)_{(j)}$ be the corresponding index of $\tau(V)$. We set the largest penalty factor (the penalty factor of the least important feature) to a fixed value, $W_{(p)} > 1$. Then,

$$\tau(V)_{(j)} = \frac{(j - (1 - \gamma)p)_+}{\gamma p} \cdot (W_{(p)} - 1),$$

where $\gamma \in (0, 1)$ is the ReLU threshold. For this penalty form, the $(1 - \gamma)p$ most important features receiving a penalty of 0, and the rest receive a positive penalty that is a function of their position in the LLM-derived feature ranking.

$W_{(p)}$ is chosen to be an arbitrary fixed value,⁵ and the elements of the family differ in their values γ .

B.2 Penalty Factor Simulations

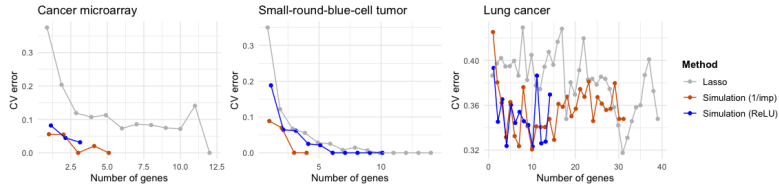


Figure 8: Test error in simulations

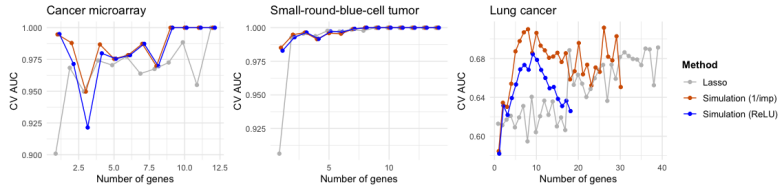


Figure 9: AUROC in simulations

We run simulations to find the adequate form of penalty factors, using datasets outlined in Table 2. Based on the simulations, we use the inverse importance penalty factors to compare the LLM-Lasso to the baseline models.

The data are split into the importance score-generating set and the cross-validation set. The hypothetical importance scores are generated by running a Lasso regression on the score-generating set

⁵Any constant scaling of the penalty factors is absorbed into the constant λ factor in Equation (4).

and assigning the absolute values of the coefficient of each feature is assigned to be the score of that feature. Then, the hypothetical importance scores are scaled so that the maximum score was 1 and the minimum score is 0.1. On the cross-validation set, we run the hypothetical LLM-Lasso using different forms of penalty factors: (i). the inverse of the importance scores and their powers and (ii). ReLU penalty factors with different thresholds. For ReLU penalty factors, we set the maximum penalty factor such that the least important feature receives a coefficient of 0 for all values of λ . We perform 5-fold cross-validation across the hyperparameter $\gamma \in (0.1, 0.2, \dots, 0.9)$ for the ReLU penalty factors and $\eta \in (0, 1, \dots, 10)$ for the inverse importance penalty factors. We obtain cross-validation misclassification rates across the spectrum of regularization parameters λ , with a $\lambda_{\min}$ to $\lambda_{\max}$ ratio of 0.01. Cross-validation is performed such that the difference in the area of the plot of the misclassification rate across numbers of features with respect to the Lasso is maximized. The best misclassification rate is obtained for each number of features selected. We perform the above for 10 data splits and plot the mean of the best misclassification rate for each number of features. The above procedure is repeated for areas under the receiver-operating characteristic curve (AUROCs) as the cross-validation metric.

Dataset	n	p
Cancer microarray [58]	52	1000
Small-round-blue-cell tumor [30]	83	1000
Lung cancer [63]	187	1000

Table 2: Summary of simulation datasets for penalty factor form.

The datasets we use in the simulations are summarized in Table 2. In the simulations, we perform the task of classifying samples into tumor tissue or healthy tissue (lung cancer dataset) or cancer subtypes (cancer microarray dataset, rhabdomyosarcoma vs others; small-round-blue-cell tumor (SRBCT) dataset, lymphoma vs leukemia) using gene expression levels. We select features with the top 1000 variances as predictors.

The simulations show an advantage of the inverse importance penalty factors over the ReLU penalty factors, as well as compared to the Lasso (Figures 8 and 9). This may be because ReLU penalties only use the order of the importance scores, which is less rich information than the inverse importance penalties. Thus, in the experiments, we use the inverse importance penalty factors to compare the LLM-Lasso to baseline models.

C Prompt Construction

Prompting is shown to be significant to the performance of LLMs. In this section, we detail the prompting strategies we explore in generating the penalty factors.

As we recall in Section 3.2.1 that our full prompt follows the following structure:

$$\mathcal{P}^{\text{full}} = \text{prompt}(\mathcal{Q}^{\text{user}}(\mathcal{A}(\phi, c)), \mathcal{C}^{\text{retriever}}(k, \mathcal{R}(\phi, c)), \mathcal{H}^{\text{system}}).$$

The design choice for the user therefore primarily resides in (i). the construction of the task description prompt $\mathcal{A}(\phi, c)$ and (ii). the construction of a customized retrieval prompt $\mathcal{R}(\phi, c)$ in the case when RAG is used.

Throughout our large-scale experiments with the biomedical datasets, we set the system message to the generation LLM as “assistant,” with instruction: “you are an expert assistant with access to gene and cancer knowledge.”

C.0.1 Task Description

The general format of text description follows Figure 2. However, there are many ways one can format each of the three sections, that is, background description, a task description, and formatting rules. In the following, we go through each component in depth.

Background Description. The background description includes the following key elements:

- *Meta-data of the dataset:* This includes details on how the data is collected, number of samples, and number of features.
- *User Intention:* This includes a description of our goal for data analysis. For conducting classification experiments using LLM-Lasso, for instance, we remark: “We wish to build a statistical (Lasso) model that classifies samples into category diffuse large B-cell lymphoma (DLBCL) and follicular lymphoma (FL).”

Task Description. The task section specifies the exact request made to the generation LLM. For LLM-Lasso, this involves a description of the penalty factors. As penalty factors can be less intuitive to understand than the straight-forward importance scores, through our experiments, we experimented with a number of prompts to describe to the LLM the meaning of “penalty factors” in an effort to boost prediction performance by facilitating understanding. To this end, we employ four prompting strategies—Bayesian, ReLU, adversarial, and empirically-calibrated—to guide the interpretation of the penalty factors. We found that this part of the prompt has a direct and considerable impact on the predictive ability of the LLM.

I. The Bayesian Approach for Prompt Construction. The Lasso with penalty factors can be interpreted from a Bayesian perspective, where the penalty factor serves as the scaling parameter of a Laplace prior. A larger penalty factor results in a tighter distribution around zero, encouraging sparsity. Under this framework, the corresponding prompt for the oncology prediction task is:

I would like you to provide penalty factors greater than or equal to 0 to use on each coefficient of a Lasso estimator based on domain knowledge for a regression or classification task. Suppose β_k is the regression coefficient for feature k . We interpret Lasso with penalty factors λ_k as yielding a maximum a posteriori estimate under Laplace priors with parameters λ_k .

This means that, before observing the data, the ratio of log-tail probabilities $\log P(\|\beta_i\| > t) / \log P(\|\beta_j\| > t)$ is equal to λ_i / λ_j for each i, j and for all t . Therefore, the penalty factors represent relative log-tail probabilities of coefficients. For example, if feature A has a penalty factor of λ and feature B has a penalty factor of 2λ , this implies that the log-likelihood of the absolute value of the regression coefficient for A exceeding any threshold is twice that of B . Thus, the larger the penalty factor for a coefficient, the less “important” the coefficient is.

II. The ReLU-form Approach for Prompt Construction. Another prompting framework for interpreting the penalty factor is to directly describe the process which we code (using either Python or R) our underlying Lasso model with penalty using the ReLU-form penalty.

We plan to use your scores with a Lasso-regularized multinomial classifier, implemented via the R package `glmnet`. The scores will generate penalty factors (weights on the ℓ_1 norm), which will be used in `glmnet`. Higher importance genes will be assigned smaller penalty factors, while lower importance genes will receive larger penalty factors.

Let `xall` denote the feature matrix (number of observations by number of genes) and `yall` the multinomial class outcome. Similarly, let `xtest` and `ytest` be the test set feature matrix and class outcome, respectively.

Let `scores` be the p -vector of gene importance scores provided by ChatGPT.
The details of our plan are implemented in the following R (Python) code: [omitted]

III. The Adversarial Approach for Prompt Construction. The penalty factor can also be interpreted as part of an adversarial game to enhance out-of-sample prediction robustness. Here, the penalty factor scales the cost of perturbing covariates under a weighted ℓ_∞ norm. Larger penalty factors make changes to a covariate more “expensive,” limiting adversarial alterations, while smaller factors make them cheaper, reflecting lower importance. The adversary operates within a fixed budget, distributing total weights across covariates to balance importance and vulnerability.

You are tasked with helping perform a what-if (adversarial) analysis to improve out-of-sample prediction on a logistic regression model for classification. Here is how this analysis works: (1). For each sample, every covariate (gene expression level) can be modified (increased or decreased), but the cost of changing each covariate is scaled by a weight that we assign now. (2). The “size” of a change to a single sample is measured by the weighted L_∞ norm: if δ_i is the change to covariate i , and w_i is the weight for covariate i , then the size of the change is: $\max_i(|\delta_i| \times w_i)$. Across the dataset, the average of these sizes is constrained by a fixed budget. (3). A larger weight on covariate i makes changes to that covariate more “expensive” to the adversary, limiting how drastically it can be altered under the same overall budget. A smaller weight makes it cheaper to perturb that feature, which might be acceptable if the gene is less important. (4). You must distribute a total of 100 weight units among all covariates: $\sum_i w_i = 100$.

Given this setup, your job is to choose weights for each predictor. Your goal is to provide a plausible weighting scheme that balances the importance of each predictor against potential adversarial changes.

IV. The Empirically Calibrated Approach for Prompt Construction. In addition to using different theoretical angles to explain to the generation LLM the notion of penalty factors, we construct a prompt based on features of the LLM-generated penalty factors that empirically result in better LLM-Lasso performance. In addition, we consult advanced LLMs, such as o1 from OpenAI for advice on tuning this prompt in a way that would be most conducive for LLM performance.

Assign each gene on the following list a penalty score between 0.1 and 1 based on its importance in distinguishing “{category}”. A lower penalty (e.g., 0.1) indicates a gene that is highly predictive for differentiating “{category}”, while a higher penalty (e.g., 1) denotes relatively minor importance for distinguishing these two subtypes.

Focus only on established evidence or strong biological rationale, rather than speculation, and be mindful that only a limited subset of genes is truly predictive for this distinction. Expect most genes to receive a score close to 1, and very few genes (maybe none) to receive a score close to 0.1. If a gene is only generally relevant to {broader_topic} and does not specifically help distinguish “{category}”, assign a penalty closer to 1.

This prompt is constructed to address several factors that can contribute to low-quality LLM scores, including reliance on speculation rather than known biological associations, over-assignment of low penalties, and assignment of scores based on broader relevance rather than the specific classification task. We note that the statement, “expect most genes to receive a score close to 1” is specific to high-dimensional cases where only a subset of features are predictive. For tasks, such as the small-scale datasets, where most features are relevant, this descriptions should be omitted. In addition, the range (0.1, 1) is arbitrary and can be chosen by the user. In fact, we find that post-processing these scores to lie within a range farther away from 0 helps downstream Lasso performance, as discussed in Appendix E.3.

The empirically calibrated prompt simpler than the other three approaches and focuses on direct instructions (i.e, more important features should be penalized more) rather than attempting to explain the intuition behind the penalty factors. While the empirically calibrated prompt seems to introduce no in-depth understanding of penalty factors that the shallow level, empirically, we find that this prompt consistently encourages better prediction performance across a range of LLMs from simple to advanced on penalty factor production for feature selection. Overall, our empirical findings suggest that: Empirically Calibrated prompt > Bayesian prompt > ReLU prompt > Adversarial prompt, in descending order of performance.

Output Format Instructions. Our experiments revealed that selecting appropriate output format instructions is crucial not only for the accuracy of the score collection process but also for maintaining the quality of the scores produced. This is especially important for smaller models with fewer parameters (e.g., llama-3-8b-instruct), which often struggle to follow prompt instructions and understand the concepts and guidance provided.

In practice, we found that directly using text responses and providing LLMs with clear text formatting rules is more effective in regulating their behavior and ensuring a smooth score collection process compared to requesting responses in raw JSON format, as commonly used in LangChain’s pipeline. For all LLMs, we attach a format instruction to the end of every prompt, with slight modifications

tailored to the specific task. Below is an example of a format instruction used for the task of outputting penalty factors for gene selection in cancer or lymphoma prediction.

Formatting Rules:

1. **Score Representation:** Use a direct floating-point number (e.g., 0.5). Avoid scientific notation (e.g., $10^{*(-2)}$ or $1e-2$) and additional formatting.
2. **Include All Genes:** Assign a penalty factor for every gene in the input list, preserving the order of input.
3. **Reasoning:** After each penalty factor, add a concise reasoning about the gene’s role in predicting {category}.
4. **Consistency:** Ensure uniform formatting. Example:

AASS: 0.15

Reasoning: This gene is highly expressed in cancer pathways and has been associated with {category}. Assigned a low penalty factor.

BRCA1: 1

Reasoning: BRCA1 is not significantly relevant for {category}. Assigned a high penalty factor.

Do not include disclaimers about lacking full data; rely on general cancer genomics and pathway relevance.

As outlined in the formatting prompt, three strategies were found to be particularly effective:

1. *Highlighting common errors:* We include a list of frequent formatting mistakes made by LLMs, identified through trial and error. These include, for example, using scientific notation instead of floating-point numbers, which complicates the score collection algorithm, and applying inconsistent additional formatting to the scores.
2. *Providing examples:* Examples demonstrating the desired score and explanation format significantly improve the LLMs’ understanding of the task. This is particularly important when querying for penalty factors instead of importance scores. While a dedicated prompt explains the concept of penalty factors, smaller models like llama-3-8b-instruct often struggle with the counterintuitive nature of penalty scores—where lower values indicate higher significance and vice versa. Including examples of both low and high penalty scores helps address this challenge and ensures better compliance.
3. *Using a firm tone:* We employ strict language to enforce adherence. Commands such as “Do not say that it’s not possible...” and “Responses not following these guidelines will be considered invalid” have proven effective in ensuring LLMs behave consistently and follow the guidance provided.

Figure 10 is an example of the full user prompt used in the study of classifying patients into DLBCL and FL, which employs o1-generated explanation of penalty factors.

C.0.2 Retrieval Prompt

The default pipeline in Langchain for retrieval query is to perform semantic similarity search on the user’s original prompt to the generation LLM. This becomes problematic, however, when the main user prompt is large and overshadows the important information that sheds light on what documents should be retrieved. As an example, when passing in directly the full user prompt for retrieval in the oncology classification tasks, semantic similarity search retrieves information on the description of the dataset, for example, contexts regarding cfDNA fragmentation pattern and EPIC-Seq, instead of what we are actually curious about, that is, the relevance of certain gene, say AASS, with classifying lymphoma subtypes, say, diffuse large B-cell lymphoma (DLBCL) and follicular lymphoma (FL).

In order to pinpoint the retriever to the relevant retrieval documents, we use a customized retrieval prompt. For the oncology classification tasks, due to the high-dimensional nature of the dataset, we batch process the genes (see Appendix E for discussion) and use the following prompt that takes each gene $g_i \in \{g_1, \dots, g_B\}$ in each batch of size B and the target classification category c :

Retrieve information about gene {g}, category {c}, especially in the context of {g}’s relevance to {c}.

***Context*:** We have gene expression data derived from cancer patient samples (cfDNA fragmentation pattern, EPIC-Seq). The dataset includes 161 samples and 1592 genes. We wish to build a statistical (Lasso) model that classifies samples into the category "diffuse large B-cell lymphoma (DLBCL) and follicular lymphoma (FL)."

****Task**:** Assign each gene on the following list a penalty score between 0.1 and 1 based on its importance in distinguishing "diffuse large B-cell lymphoma (DLBCL) and follicular lymphoma (FL)". A lower penalty (e.g., 0.1) indicates a gene that is highly predictive for differentiating "diffuse large B-cell lymphoma (DLBCL) and follicular lymphoma (FL)", while a higher penalty (e.g., 1) denotes relatively minor importance for distinguishing these two subtypes.

Focus only on established evidence or strong biological rationale, rather than speculation, and be mindful that only a limited subset of genes is truly predictive for this distinction. Expect most genes to receive a score close to 1, and very few genes (maybe none) to receive a score close to 0.1. If a gene is only generally relevant to lymphoma and does not specifically help distinguish "diffuse large B-cell lymphoma (DLBCL) and follicular lymphoma (FL)", assign it a penalty closer to 1.

The penalty factors must be listed in the exact same order as the order of genes provided above. Each penalty factor must immediately follow the corresponding gene name after a double-asterisk-colon (**:**) and adhere to the following format:

- ****GENE NAME**:** VALUE (float) - Example: ****AASS**:** 2

****Instructions**:**

1. You will receive a list of genes: {genes}.
2. For each gene, produce an floating point penalty factor from 0.1 to 1.
3. List the genes and their penalty factors in the exact same order they appear in the list.
 - Note that letter "I" is not number "1", so do not write "ARSI" as "ARS1".
 - For each gene, include ALL its letters: for instance, "BYSL" is NOT "BYS".
4. For each penalty factor, provide a brief statement of how you arrived at that factor or why the gene is more or less relevant to "diffuse large B-cell lymphoma (DLBCL) and follicular lymphoma (FL)."

Do not include disclaimers about lacking full data; rely on general cancer genomics and pathway relevance.

The list of genes is ["AASS", "ABCA6", "ABCB1", "ABHD6", ...].

Figure 10: Example of a full user prompt for experiment study DLBCL vs FL.

An example prompt using this format is as follows:

Retrieve information about gene AASS, category "transformed follicular lymphoma (tFL) and follicular lymphoma (FL)", especially in the context of AASS's relevance to "transformed follicular lymphoma (tFL) and follicular lymphoma (FL)".

For each pass of retrieval search with gene and lymphoma pair, we retrieve top k relevant documents. After collecting the contexts for all the genes in the batch, we then filter for the unique documents and then append them to the full prompt in prompt component $\mathcal{C}$. It turns out that the specific implementation of this procedure is an art: we want to strike a balance between overwhelming the generation LLM with long-context and potentially minimally informative documents and excessive cautious retrieval that does not inform the LLM by much.

D Experiment Supplement

D.1 Model Details

We provide more details of the LLMs used. Table 3 summarizes the cut-off dates in each LLM.

Model Name	Company	Cut-off Date	Source
GPT-3.5 (Turbo)	OpenAI	2021.09	Source
GPT-4o (2024-08-06)	OpenAI	2023.10	Source
o1	OpenAI	2023.10	Source
Llama-3-8B	Meta	2023.12	Source
Llama-3.1-405B	Meta	2023.12	Source
DeepSeek-R1	DeepSeek	2024.07	Source
Qwen Models-72B	Alibaba	2023.09	Source

Table 3: Surveyed LLMs Cutoff Dates Overview

D.2 Dataset Details

In this section, we give more details on the datasets used in the Experiment Section.

D.2.1 Small-scale Experiment Datasets

We source a wide range of small-scale datasets for feature selection in classification and regression. We use * to indicate the datasets that are released after the cutoff dates for all models sampled (see Table 3 for an overview of the model cutoff dates). For all small-scale datasets, we remove features whose values are not numerical and not categorical and remove rows and columns with missing values. We remark that the purpose of the small-scale experiment is not meant to demonstrate performance on the specific task but rather to show case the ability of the feature selector candidate, even in the absence of some potentially informative features and data.

Dataset	Year	n	p	Source
Spotify*	2024	4600	29	Source
Wine	2009	6497	11	Source
Diabetes	1998	768	8	Source
Bank	2012	45211	51	Source
Glioma	2022	839	23	Source

Table 4: Summary of small-scale experiment datasets.

D.2.2 Large-scale Experiment Datasets

The datasets used in the large-scale experiments are outlined in Table 5, where n, p denotes resp. sample size and number of features. We note that the lymphoma dataset remains unpublished and is currently confidential (the study in which this dataset is collected has been approved by the Stanford Institutional Review Board #13500 and #25216). Although we cannot disclose its full details, we provide additional context below regarding the background of this dataset and its clinical relevance for lymphoma classification.

Lymphomas are hematological cancers arising from lymphocytes that can broadly be categorized into Hodgkin lymphomas (HL) and non-Hodgkin lymphomas (NHL). NHL tumors tend to be more diverse and heterogeneous in their subtypes, growth rate, therapeutic responses, and urgency. The most common NHL subtypes include DLBCL (Diffuse large B-cell lymphoma), FL (Follicular lymphoma) and MCL (Mantle Cell lymphoma). Several lymphoma subtypes including FL, MCL, and HL can morphologically transform into DLBCL, and this transformation event can often be fatal. For example, among patients initially diagnosed with slow growing, indolent FL tumors that can be expectantly observed without treatment, a significant minority experience aggressive histological transformation to DLBCL requiring immediate therapy, with detection of this phenomenon often posing clinical and diagnostic challenges. Similarly, while HL typically has a favorable prognosis compared with many NHL counterparts, transformation of HL to DLBCL can introduce significant diagnostic and therapeutic problems. Therefore, to validate our LLM-Lasso framework for the classification of HL/NHL subtypes, we study an unpublished dataset profiling samples from 287 patients with DLBCL (100), FL (30), MCL (61), and HL (96) for the expression of 1592 genes.

Dataset	n	p
Lymphoma (FL vs DLBCL)	130	1592
Lymphoma (MCL vs DLBCL)	161	1592
Lung cancer (LUAD vs LUSC)	1017	1000

Table 5: Summary of large-scale experiment datasets.

D.3 Supplemental Experiment Results

D.3.1 Ablation of Model Temperature

We perform ablations over LLM temperature with $\text{temp} = \{0, 0.5, 1\}$, on the MCL vs. DLBCL lymphoma classification task. We find that the test error is agnostic to the temperature for both plain and RAG-augmented LLM-Lasso. Results are plotted in Figures 11a and 11b. The results for plain and RAG-enhanced LLM-Lasso are very similar, with lower test error for RAG.

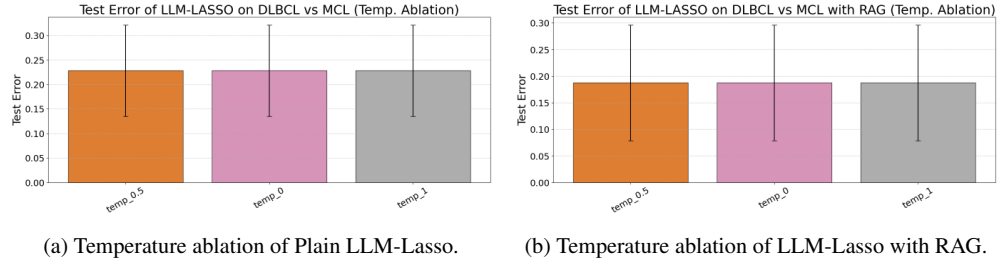


Figure 11: Temperature ablations of LLM-Lasso, where error bars are standard deviation over 10 different training and test splits, computed via `pandas DataFrame` aggregation. Although plotting standard deviation error bars assumes normally-distributed errors, this visualization suffices for our purposes of showing that the temperature has little impact on LLM-Lasso performance.

D.3.2 Feature contributions

In the experiments we run multiple LLM-Lasso regressions, and thus we are unable to extract a single list of selected features and their coefficients. For better interpretability, we introduce a feature contribution metric that takes the proportion that each feature appears across the full path of the number of features. A feature contribution of 1 means the feature appeared in all the models, while that of 0 means the feature appeared in none of the models. We create heatmaps of the union of genes with top 10 feature contributions for the Lasso, Plain LLM-Lasso, and RAG LLM-Lasso, as well as the polarity of the coefficients, represented as letters in the heatmaps (“F” coefficients in the direction of FL and “D” for DLBCL) (Figure 12).

In the clinically relevant problem of classifying FL and DLBCL, there are several genes with high feature contributions that have relevance in cancer genomics and hematology/oncology, especially in the GPT-4o LLM-Lasso heatmap. For example, *AICDA*, *BCL2*, and *BCL6*, all of which have high feature contributions in the RAG LLM-Lasso, have been implicated in the transformation of FL to DLBCL [43, 21]. Consistent with our findings, a previous study suggests that high *AICDA* expression is implicated in the generation of mutations in the *BCL2* gene, which was associated with increased risk of the transformation of FL into DLBCL [12]. In addition, higher expression of *MYC* has been implicated in the transformation of FL to DLBCL [2], which is concordant with the identification of *MYC* as an important feature to classify DLBCL from FL by the LLM-Lasso. On the other hand, plain Lasso does not consistently select such genes.

Interestingly, the o1 LLM-Lasso heatmap, although *AICDA* is included as the top gene in the RAG LLM-Lasso, many of the other genes are less relevant to the DLBCL literature [53]. The reason for this contrast with the high accuracy of o1-based LLM-Lasso is unknown; further investigation may be of interest.

D.3.3 Deferred Plots

In this subsection, we present the deferred plots from the main experiment section (Section 5).

Figure 13 provides an illustrative example of LLM hallucinations in the adversarial experiments, where GPT-4o produces seemingly-plausible explanations for scores assigned to fake genes.

Figure 14 illustrates the AUROC performance of our model against various baseline across the three high-dimensional lymphoma datasets. It is evident that the strong performance demonstrated by Figure 5 carries over to the AUROC metric.

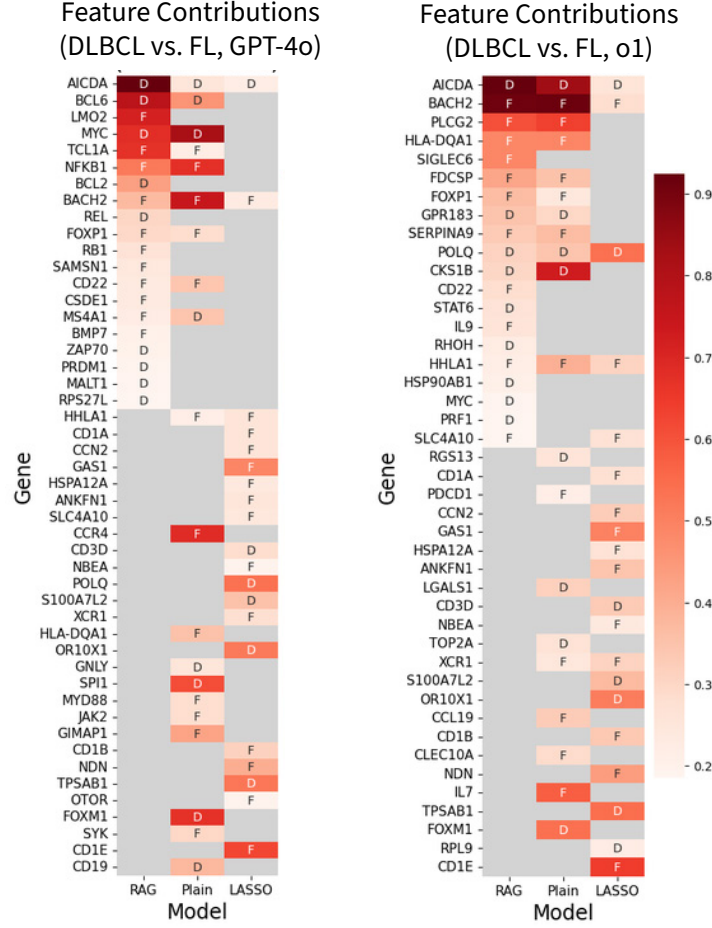


Figure 12: Heatmaps of feature contributions in the FL vs DLBCL experiment for GPT-4o and o1 LLM-Lasso, with and without RAG.

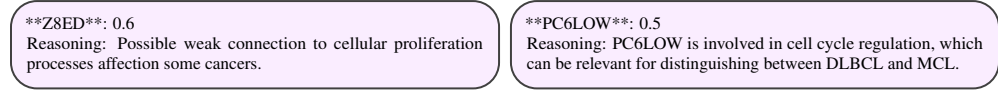


Figure 13: GPT-4o hallucination for corrupted gene names: an LLM-Lasso penalty factor (left) and an LLM-Score importance score (right). Even though both genes, Z8ED and PC6LOW are fake, the LLM hallucinates justification for their relevance to the task.

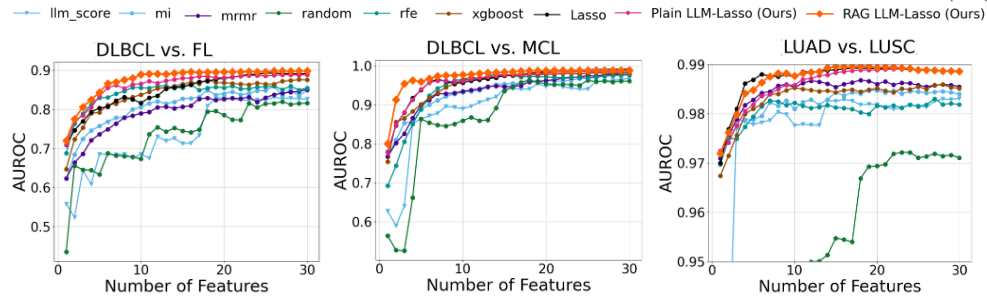


Figure 14: AUROC performance across 10 splits for the lymphoma datasets.

Figure 15 shows a close up of the Lasso and RAG LLM-Lasso lines from the large-scale experiments (Figure 5), highlighting the accuracy improvement from adding LLM-generated penalty factors.

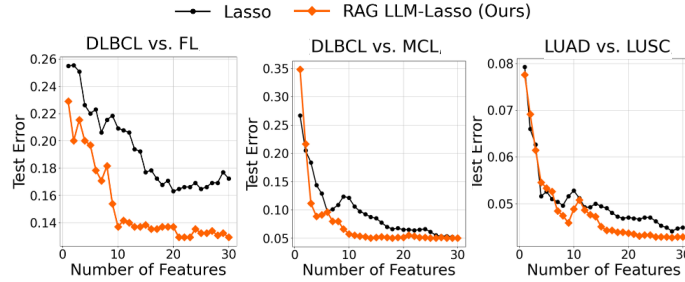


Figure 15: Comparison of Lasso vs. RAG-enhanced LLM-Lasso on various oncology datasets.

Figure 16 describes the model ablation study on the Spotify regression dataset for feature selection. As we can see, the GPT-4o model we used in Figure 4 is the top performing model, but all models except for the smaller LLaMa-3-8b-instruct model beat the Lasso baseline. Consistent with Figure 5, performance is more or less correlated with model size, though the trend is not definitive.

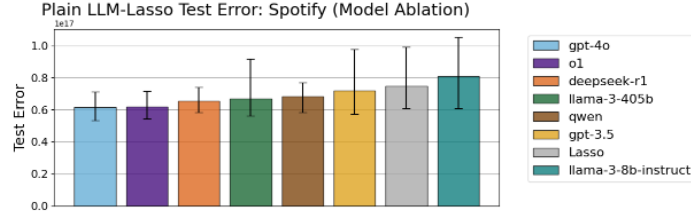


Figure 16: Model ablation on the Spotify dataset with test error computed at $\approx 25\%$ of total features. Error bars are 0.95 and 0.05 quantiles over 10 training and test splits, computed via the `pandas.quantile` function.

D.3.4 Discussion: Impact of RAG

As shown in the misclassification error plots of Figure 5, LLM-Lasso RAG outperforms the plain LLM-Lasso. Appendix D.3.4 provides an illustrative example justifying this, in which a RAG-enhanced GPT-4o model cites concrete gene interactions. RAG, however, does not unilaterally improve performance. This can result from the knowledge base being ill-suited to the task, issues with long contexts, and retrieval of irrelevant information.

To illustrate the benefits of RAG, we query GPT-4o with “Can you concisely tell me about the gene BCL6?” twice: once without RAG, and once with the OMIM RAG pipeline described in Section 5.4.2. The corresponding outputs are displayed in Figure 17. Without RAG, the response is more generic, whereas the RAG-enhanced response is more detailed and scientific, specifically describing how BCL6 is related to lymphoma and citing interactions with proteins and other genes.

E Implementation Details

E.1 Computational Resources

The LLM score-collection process requires a number OpenAI or OpenRouter queries equal to the number of batches of features, times the number of trials. The large-scale experiments perform score collection in batches of $\approx \sqrt{n}$ genes, requiring ≈ 40 batches for each large-scale experiment. The experiments using the o1 model are performed with 3 trials, and the experiments using GPT-4o are performed with one trial (as the temperature parameter of the LLM was set to 0). Scores are collected with 10 threads performing API calls in parallel, though this is only an optimization to reduce experiment runtime. Model ablation studies only used one trial regardless of model type (with temperature set to 0 where allowed), due to the large volume of LLM queries needed otherwise. The

Prompt

Can you concisely tell me about the gene BCL6?

Without RAG

BCL6 (B-cell lymphoma 6) is a gene that encodes a transcriptional repressor protein involved in the regulation of immune responses. It plays a critical role in the development and function of B cells, particularly in the formation of germinal centers where B cells mature and undergo antibody diversification. BCL6 is crucial for processes like somatic hypermutation and class switch recombination. Abnormal expression or translocations involving BCL6 are associated with certain types of lymphomas, especially diffuse large B-cell lymphoma (DLBCL). As a transcriptional repressor, BCL6 regulates the expression of genes involved in cell cycle control, apoptosis, and differentiation.

With RAG

The BCL6 gene encodes a 79-kD protein that functions as a zinc finger-transcription factor, primarily involved in B-cell differentiation. It is predominantly expressed in the B-cell lineage, specifically in mature B cells and germinal centers. BCL6 is implicated in the pathogenesis of certain B-cell lymphomas, particularly diffuse large B-cell lymphoma (DLBL), where its expression may be deregulated due to chromosomal translocations affecting the 3q27 region. These translocations can lead to fusion with other genes, such as the Ikaros gene, potentially contributing to lymphoma development by preventing normal B-cell differentiation.

Figure 17: Output of GPT-4o responses, with and without RAG.

LLM-Score baseline requires the same number of queries as LLM-Lasso, and LMPriors requires one query per feature. In total, the large-scale experiments require about 840 queries (including plain LLM-Lasso, RAG LLM-Lasso, and LLM-Score), the small-scale experiments use 132 queries (mainly from LMPriors), and the model ablations use about 300 queries. The adversarial experiments require 80 queries.

Populating the OMIM vectorstore via Chroma takes around 5 minutes on a MacBook M1 10-core CPU. For RAG-augmented LLM Lasso, querying the OMIM RAG database for the 1592-gene dataset takes about 10 minutes per experiment, resulting in 30 minutes for the large-scale experiments and 70 minutes for the model ablations.

The Lasso component of LLM-Lasso, as well as the baselines, are run on a laptop with 16 AMD Ryzen 9 5900HX processors and 16 GB of memory. Lasso and logistic regression models are run using 8 threads (implementation details of which are handled by `adelie` and `scikit-learn`, respectively). The data-driven baseline feature selectors take approximately 30 minutes for each large-scale experiment and 2 minutes for each small-scale experiment, resulting in 100 total minutes. Each downstream experiment (LLM-Lasso and the downstream logistic regression model for the baselines, for a single dataset) takes approximately 15 minutes for experiments with multiple trials of LLM prompting and 10 minutes otherwise. This results in about 35 minutes total for the large-scale experiments, 50 minutes total for the small-scale experiments, and 10 minutes for the adversarial experiments. The model ablations do not require running all of the baselines (only LLM-Lasso and Lasso), and they take approximately 15 minutes total.

E.2 Handling Token Limits in LLMs

Input and output token limits in closed-source pretrained LLMs pose significant challenges, especially when querying large sets of predictors, as they restrict the information processed or returned in a single interaction. The absence of memory retention further complicates output aggregation. This limitation affects both closed-source GPT models via the OpenAI API and cloud-hosted open-source models, which also lack persistent memory.

However, ensuring that an LLM has sufficient output tokens is critical for its performance. For instance, [71] showed that step-by-step reasoning improves effectiveness, and we observe that LLMs struggle under tight token limits or when limits are exceeded. To handle large feature sizes, batch-querying with an appropriate batch size is necessary to stay within token limits. However, this approach introduces challenges: without memory retention, the LLM cannot access previously processed features or their scores, leading to inconsistencies when aggregating batch results and potential scale mismatches. To address these issues and ensure accurate feature diagnoses while preserving essential output tokens, we propose three strategies that require no fine-tuning or parameter modifications.

Constrained Scores. The most simple way to address this challenge is to constrain the penalty factors to be in a pre-determined range, which we encode in our prompt (see Figure 2 and Appendix

E.3). In this case, a score of, e.g., 0.3, corresponds to approximately the same degree of importance across batches. We set this range to be between 0.1 and 1 for the large-scale datasets and between 2 and 5 for the small-scale datasets. If the range is too large, some models (e.g., GPT-4o) provide penalty factors too close to the extremes. This can lead to convergence issues in the downstream algorithm, and increase the impact of spuriously low or high scores.

Text-based Summary. Another straightforward approach is to batch-query the features while enabling memory retention in the LLM by augmenting the user query Q^{user} with a summarization of chat history $\mathcal{H}^{\text{system}}$, stored in a conversation buffer constrained by the max token limit. Several open-source Python packages support this functionality. In our implementation, we use LangChain’s `ConversationBufferMemory`. While not ideal for score-collection scenarios—since summarization often omits full scores and context due to token constraints—we find that including memory increases the likelihood of the LLM assigning scores on a consistent scale and provides marginal improvements in prediction performance.

Statistical Estimation. Finally, the issue can be addressed via statistical techniques to infer the true score from batch scores without injecting memory into each batch. We introduce the following method. We note that to balance batch size and the number of queries, we heuristically select a batch size of $\lceil \sqrt{p} \rceil$, where p is the total number of features and $\lceil \cdot \rceil$ is the ceiling notation.

Scaling. Given batch scores $B_1(s), \dots, B_{\lceil \sqrt{p} \rceil}(s)$, the scaling method involves selecting the maximum score from each batch, $s_{\max,1}, \dots, s_{\max,\lceil \sqrt{p} \rceil}$, and passing these maximum scores as a new batch to the LLM for rescaling, yielding $\tilde{s}_{\max,1}, \dots, \tilde{s}_{\max,\lceil \sqrt{p} \rceil}$. The final score is then computed by weighting and concatenating the batch scores. Specifically, each batch B_i is weighted by $\frac{\tilde{s}_{\max,i}}{\sum_{j=1}^{\lceil \sqrt{p} \rceil} \tilde{s}_{\max,j}}$, which is the normalized rescaled maximum candidate from that batch relative to the rescaled maximum candidates across all batches.

Choice of Strategy. For the results in this paper, we use constrained scores and text-based summary. Further exploration of the statistical estimation technique is left for future work.

E.3 Evaluation Methodology: Prediction Performance

For the large-scale experiments, factors are collected using the empirically-calibrated prompt. As the empirically-calibrated prompt is tuned for high-dimensional problems where only a small percentage of features are predictive, it is not relevant to the small-scale experiments. For the small-scale experiments, we instead use the following task description:

Provide penalty factors for each of the features. These penalty factors should be integers between 2 and 5 (inclusive), where: 2 indicates a feature strongly associated with the target variable (i.e., it should be penalized the least by Lasso). 5 indicates a feature with minimal relevance to the target variable (i.e., it should be penalized the most by Lasso). Focus only on established evidence or strong rationale, rather than speculation. Focus on features that are immediately predictive for the task at hand, rather than those that are more generally relevant.

We also include a short description of each feature in the small-scale experiments, placed at the end of the prompt right before the full list of features.

For the large-scale experiments, o1 is used for DLBCL vs. FL and LUAD vs. LUSC, and GPT-4o is used for DLBCL vs. MCL. Scores are collected in batches of 40 genes for the lymphoma dataset and 30 genes for the lung cancer dataset. The small-scale experiments all use GPT-4o, and all scores are collected in a single batch.

To test the prediction performance of the LLM-Lasso, the data is centered and split into the training set and the test set. On the training set, we perform 5-fold cross-validation across the hyperparameter $\eta \in (0, 1, 2, \dots, \eta_{\max})$ for penalty factors of the inverse importance form, V^η .⁶ As in the simulations

⁶The specific value of $\eta_{\max}$ varies per dataset. For the small-scale experiments, is 4 for Diabetes and Spotify, 2 for the remainder of the small-scale experiments. For the GPT-4o large-scale experiment (DLBCL vs. MCL), $\eta_{\max} = 4$, and it is 1 for the o1 experiments (to reduce the overhead of the cross-validation step).

(Appendix 4), the cross-validation loss function is the negative difference in the area of the plot of the misclassification rate across numbers of features with respect to the Lasso.

Using the set of scores and transformation with the lowest cross-validation misclassification rate, we evaluate model performance (RAG LLM-Lasso, plain LLM-Lasso, and baselines) on the test set. For each number of selected features, we record the misclassification error and AUROC for the model with the best cross-validation error.⁷ We repeat the process across 10 random splits and plot the mean.

Additional Heuristics For the large-scale experiments with o1, where we are not permitted to set the LLM generation temperature to $T = 0$, we collect scores across 3 trials and use cross-validation to select the best set of scores.

For the empirically-calibrated prompt, we find that the range of the scores is not conducive to optimal Lasso performance. Accordingly, we add a constant factor of 2 to each penalty factor before applying the inverse importance transformation (placing the penalties in the $[2.1, 3]$ range).

As we find that ℓ_2 regularization is essential for high test accuracy for low numbers of selected features, we run downstream ℓ_2 -penalized logistic regression on the LLM-Lasso- and Lasso-selected features for up to 5 features in the small-scale experiments and up to 10 features in the large-scale experiments.

E.4 Score Collection

LLMs, especially smaller models can make formatting mistakes. For instance, they may not including all necessary genes, or may include extra genes (e.g., ones mentioned in retrieval context) as part of the genes to score. For OpenAI models, we use structured outputs to directly receive the scores as a Python object. In models where this streamlined score collection feature is not available, we rely on the output formatting from the prompt (see Appendix C) and search for floating point scores that immediately follow the double-asterisk-colon sign (with or without space). If we fail to collect scores, we retry until the correct scores for the batch are collected.

E.5 R implementation

Once the importance scores are obtained from the LLM, the LLM-Lasso can be implemented in R [56]. One can pass the penalty factors, transformed into the form of choice, such as the ReLU-form or inverse importance and their powers ($\mathcal{I}^{-\eta}$), into the `cv.glmnet` function in the package `glmnet` [19]. The penalty factors can be passed into an argument called `penalty.factor`, which specifies the penalty factors to be assigned to each feature.

E.6 Python Implementation

The full end-to-end pipeline of LLM-Lasso is implemented in Python. The score collection is done via OpenAI APIs for GPT models and o1, and via OpenRouter otherwise. Langchain is used for the retrieval component of RAG.

For computing the data-driven baseline metrics (such as mutual information and MRMR), we first produce a set of randomly generated 50/50 train and test splits and save them to CSV files. These splits are used for both LLM-Lasso and each data-driven baseline. Then, baseline scores can be computed via our Python implementations, relying on `scikit-learn`.

The implementation of the LLM-Lasso model, given the importance scores, is based on the package `adelie` [74]. We have a custom fork of `adelie` that adds AUROC and misclassification error metrics to the output of `adelie.cv.cv_grpnet`. For the transformation applied to the penalty factors, we consider powers of the inverse importance. Cross-validation, with folds determined by `scikit-learn`'s `StratifiedKfold`, determines which power of the inverse importance to use. Results are averaged across the same folds as used to compute the baselines.

For more details, refer to the code submission.

⁷For regression, the mean squared error is used instead of misclassification rate.

F Impact Statement

This paper contributes to the advancement of machine learning and statistics by improving the robustness of LLM-based feature selection. By reducing susceptibility to overfitting, our approach enhances the reliability and generalizability of feature selection methods, improving the trustworthiness and interpretability of AI. Our method is generic: it can be applied to any tabular dataset whose features have meaning. In particular, it brings advancements in the biomedical domain, especially for high-dimensional datasets such as *omics* data. Our method both builds on scientific progress, by borrowing information from previous literature, and can lead to novel scientific discoveries, by fitting a model to a new dataset.

As our method relies on LLMs, it has a risk of hallucinations and disinformation. This could result in incorrect results, which may have serious societal consequences in fields such as biomedicine. For example, an ineffective drug developed based on an incorrect result from the LLM-Lasso applied to *omics* data would be a negative societal consequence. However, by design, the LLM-Lasso guards against this risk through the cross-validation procedure that determines the penalty factor form, with which the user can determine the extent to which the method relies on the scores generated by the LLM. Furthermore, the penalty factors of the LLM-Lasso is retrieved based on previous literature. Thus, it could be prone to biases against specific groups if they are present in the literature. This too can be minimized through the cross-validation procedure if the data do not contain the bias that is present in the literature.